

Sick Leave Approvals — Nothing Is Paid Until You Say So

Prepared for: Michael Lyman, Owner, Greenway Marijuana **Legal entity:** LYMAN'S MARIJUANA L.L.C. · WA UBI 603 353 555 · EIN 46-4217016 **Slice:** books-35 · **Migrations:** 0198 (tables), 0200 (split-draw index fix) **Screens:** /admin/books/leave — "Sick Leave Approvals" under Accounting /admin/staffing/clock — the request pad, at the station your staff already use **Date prepared:** 22 August 2026

The short version

You asked for Option 1: a sick day has to be **approved by you before it ever reaches a timesheet**. That is now how it works, and it is enforced in three independent places — the screen, the server, and the database itself.

An employee who wakes up ill walks to the shared station, taps a button, picks a reason and a length, and types their PIN. That creates a request marked `pending`. It changes no balance, moves no minutes, and adds nothing to any timesheet. It appears in a new inbox under Accounting. You open it, and the screen shows you the six questions the law actually requires you to answer, in the order they have to be answered, with the regulation text sitting right next to each one. You approve or you deny, in writing. Only then does anything move.

This report explains why each piece is shaped the way it is, quotes the regulations verbatim so you can check me, and — as always — tells you about the mistakes I made building it, because those are the most useful part.

Why an approval step exists at all

It would have been easier to let an employee mark themselves sick and have the hours flow straight to the timesheet. Plenty of small-business payroll systems do exactly that. Here is why that would have been the wrong build for you.

Paid sick leave is not a courtesy you extend. It is a statutory entitlement with conditions attached, and the conditions are the entire problem. Some of them are about the employee — have they been employed long enough, do they have the balance, is the reason a qualifying one. Some of them are about you — are you paying the right rate, are you paying it by the right payday, are you refraining from demanding proof you are not entitled to demand.

If the software decides all of that silently, then on the day a claim is filed, your defence is "the computer worked it out." That is not a defence. What protects you is a **contemporaneous written record of a human decision, with the reason attached**. That is what the approval step manufactures. Every approval and every denial now carries your name, a timestamp, and a note of at least ten characters, and the database will not accept a decision without all three.

There is a second reason, and it is the one that will matter more often. **The employee, not you, decides whether to use their sick leave.** WAC 296-128-630(1):

"This right means an employee has the choice about whether or not to use accrued, unused paid sick leave when a qualified purpose occurs and an employer may not require an employee to use accrued, unused paid sick leave if the employee does not choose to request to use paid sick leave."

Read that carefully, because it cuts against the intuition of most employers. You may not *push* sick leave onto someone. If an employee misses a shift and does not ask to be paid from their sick bank, you cannot help yourself to their balance to make the timesheet tidy. A system that automatically converted every absence into sick-leave usage would violate that subsection routinely, and it would do so invisibly. Requiring an explicit **request** from the employee, and an explicit **approval** from you, is the design that matches the statute: they choose to ask; you choose to answer.

What your staff see, and what it deliberately does not ask

The request pad lives on the clock screen because that is the only screen most of your people can reach. Migration 0198 recorded the reasoning at the time the tables were designed, and it is worth reproducing because it explains a security decision that would otherwise look sloppy:

"employees.staff_id is nullable 'for floor-only staff who just clock in at a shared station'. Most of Greenway's employees have no back-office login at all. If requesting sick leave required is_owner(), or even required a staff profile, then the people the statute is written to protect would be structurally unable to ask. The request would have to travel by text"

message to Michael, which is precisely the undocumented channel this migration exists to replace."

That is the whole design in one paragraph. A budtender with no email address can ask for a sick day on the same screen they already use twice a day, and **the asking is written down**. The text message to your phone at 6am — which is what happens today, and which leaves no record either of them asking or of you agreeing — is replaced by a row in a table.

Because the door is deliberately wider than usual, it carries compensating locks: the timeclock permission, then the same PIN-pad lockout the clock already uses, then the PIN itself. And **the employee's identity comes from the PIN lookup, never from the browser**. That single detail is what stops one person at a shared station filing a sick day in a colleague's name — and it would be the colleague who showed up as absent.

The pad asks four things: what kind of reason, how long, was it foreseeable, and an optional note. It does not ask what is wrong with you. That is not squeamishness, it is WAC 296-128-660(4):

"Employer-required verification may not result in an unreasonable burden or expense on the employee."

The note field is optional and the screen says so in plain words — *"You do not have to say what is wrong with you."* The domestic-violence option says outright that no further detail is needed. A required free-text box asking why would become an informal medical-verification demand, imposed on every absence regardless of length, which is precisely the burden that subsection forbids.

Two smaller decisions worth knowing about:

Length is chosen from buttons, not typed. A free number box invites an employee to type "8" meaning eight hours while the field means eight minutes. They would have asked for almost nothing, the request would look valid, and nobody would catch it. The presets are full day, half day, two hours, one hour.

"Did you know about this in advance?" defaults to no. There are two different notice rules and the record has to know which one applied. WAC 296-128-650(1)(a):

"If the need for paid sick leave is foreseeable, the employer may require advance notice from the employee. Unless the employer allows less advance notice, the employee must provide notice at least ten days, or as early as practicable, in advance of the use of paid sick leave."

And WAC 296-128-650(1)(b):

"If the need for paid sick leave is unforeseeable, the employer may require notice from the employee. The employee must provide notice to the employer as soon as possible before the required start of their shift, unless it is not practicable to do so."

It defaults to unforeseeable because waking up ill is the common case, and because that is the answer that asks *less* of the employee. Defaulting the other way would quietly stamp every sudden illness as late notice — building a paper record against your own staff by accident.

What you see: the six questions, in order

The inbox does not just show you a request and two buttons. It walks you through the review in a fixed order, because the order matters. Question two is eligibility — has this person reached their ninetieth day. WAC 296-128-630(2):

"An employee is entitled to use accrued, unused paid sick leave beginning on the 90th calendar day after the commencement of their employment. Employers may allow employees to use accrued, unused paid sick leave prior to the 90th calendar day after the commencement of their employment."

Notice the second sentence. You *may* be more generous. The system will tell you when someone is inside their first ninety days rather than silently refusing, so the choice stays yours and gets recorded either way.

Question four is the rate, and this is the one that quietly costs employers money. WAC 296-128-670(1):

"For each hour of paid sick leave used, an employee must be paid the greater of the minimum hourly wage rate established by RCW 49.46.020 or their normal hourly compensation."

The greater of. Not their normal rate. If Washington's minimum wage rises above what someone is being paid — or if their normal compensation is difficult to pin down because of tips, commissions or shift differentials — the floor is the state minimum. This is exactly why I keep asking you for the 2027 Washington minimum wage as soon as L&I publishes it (expected around 30 September 2026). It is not a nice-to-have. Until it is entered, the system cannot verify the floor, and it will say so

rather than guess.

And for completeness, the accrual rule the balances are built on, WAC 296-128-620(1):

"Employees accrue paid sick leave for all hours worked. An employee must accrue at least one hour of paid sick leave for every forty hours worked as an employee. Employers may provide employees with a more generous paid sick leave accrual rate."

Two consequences for payroll you should not have to discover in April

Sick pay is not hours worked, so it does not create overtime. 29 CFR §778.218(a):

"Payments which are made for occasional periods when the employee is not at work due to vacation, holiday, illness, failure of the employer to provide sufficient work, or other similar cause, where the payments are in amounts approximately equivalent to the employee's normal earnings for a similar period of time, are not made as compensation for his hours of employment. Therefore, such payments may be excluded from the regular rate of pay under section 7(e) (2) of the Act and, for the same reason, no part of such payments may be credited toward overtime compensation due under the Act."

In practice: someone who works 36 hours and takes 8 hours of sick leave is paid for 44 hours, but **none of it is overtime**, because only 36 were worked. A system that treated paid sick hours as worked hours would generate phantom overtime every time somebody was ill in a busy week. It would cost you real money and it would look completely normal on the payroll register.

There is a deadline for paying it. WAC 296-128-680(1):

"Unless verification for absences exceeding three days is required by an employer, the employer must pay paid sick leave to an employee no later than the payday for the pay period in which the paid sick leave was used by the employee. If verification is required by the employer, paid sick leave must be paid to the employee no later than the payday for the pay period during which verification is provided to the employer by the employee."

This is the practical argument for clearing the inbox before you approve each pay period rather than after. A request that sits unapproved past its payday is a late payment of sick leave, whatever the reason for the delay.

What I got wrong this slice

Three things, all caught before they reached you, and all instructive.

The index that made a lawful approval impossible. The ledger had a uniqueness rule allowing one usage row per request. That sounds obviously right. It is not: a single day of leave can lawfully draw from two different buckets — this year's accrual and last year's carryover — and the statute's carryover rules make that a normal event, not an edge case. The original index would have rejected the second row and the approval would have failed with a database error for a perfectly legal request. Migration 0200 widens the rule to one row per request *per source*. This is the kind of defect that only appears months in, on a real employee, at year end.

A test that was wrong about correct code. The first run of the request-pad gate came back forty green and one red — and the red was my test, not the software. I had pinned a phrase to prove that comment text cannot satisfy a test, but the phrase wraps across two lines in the file, so the check could never match no matter how correct the code was. A test that fails on good code and would keep failing after any fix is not a strict test, it is a broken one. It is fixed, and the reasoning is written into the test file so the next person does not repeat it.

A guard that did not exist. The inbox screen carried a comment claiming its regulation citations were "checked at build time against the real registry." They were not — I had written the comment describing an intention. That is the most dangerous kind of error in this codebase, because a false claim of verification is worse than no claim: it stops the next reader from checking. The check now genuinely exists, and it reads the citation IDs out of the screen's own source and resolves every one against the registry. I found it because I mistyped a citation ID earlier in the same slice and nothing complained.

How I know it works

Claiming something is tested is easy, so here is what was actually done.

The two new test suites total 126 checks. Green tests prove very little on their own, so each suite was attacked: I deliberately

broke the code fourteen different ways and confirmed the tests caught every one, restoring each file byte-identical afterwards and verifying with `diff`. The sabotages were the real failure modes, not cosmetic ones — taking the employee's identity from the browser instead of the PIN, removing the failed-PIN counter so brute force became free, checking the lockout after the PIN instead of before, deleting the domestic-violence option, making the optional note required, letting a bad date silently become today, removing the inbox refresh, and — the one that would have destroyed Option 1 entirely — letting the pad write its own request as `approved`.

Then the whole thing was run against a real PostgreSQL 15 database, not a mock. All 200 migrations applied in order. The pad's insert was executed for real and came back `pending` with no decider attached. A purpose outside the five the statute allows was rejected by the database. An approval with a two-character note was rejected. A full approval — status, your identity and the timestamp together — succeeded. And an attempt to strip the decider off an already-approved row was refused, which means an approved sick day cannot quietly lose the record of who approved it.

Repository totals now stand at 395 test files and 8,605 checks, all passing, with type checking and linting clean.

What I still need from you

Two items block work already designed and waiting:

1. **The 2027 Washington minimum wage**, as soon as L&I publishes it (expected around 30 September 2026). Without it the WAC 296-128-670(1) "greater of" floor cannot be verified, only flagged.
2. **The sick-leave accrual rate you want to use**. The statutory floor is one hour per forty worked. You may be more generous, and if you intend to be, the system needs to know before the first accrual is calculated rather than after.

And one standing item worth repeating: the fifteen inactive employees. Whether they are terminated, seasonal or simply dormant changes their carryover treatment, and that question gets harder to answer the longer it waits.

Where this sits in the build

Done: company setup, the tax tables, deposit scheduling, timesheets and overtime, the sick-leave engine, the garnishment engine, and now the approval workflow that connects an employee's request to your decision to the timesheet.

Next: net pay — the calculation that takes gross wages, applies withholding, subtracts garnishments in their lawful order, and produces the number on the cheque. That slice also builds the year-to-date store, which is the piece every quarterly and annual form depends on.

After that, by your instruction, I will step back and give you the full CPA inventory: everything an enterprise-grade payroll system should have, measured against what we have built, with the gaps ranked by how much they can hurt you.